

Conectando la Universidad de la Habana (UH)

Asignatura: Diseño y Análisis de Algoritmos

Jabel Resendiz Aguirre

Noel Pérez Calvo

Arianne Camila Palancar Ochando

Enero 2026

- 1 Algoritmo Fuerza Bruta
- 2 Algoritmo Heurístico: Reducción del Árbol de Oportunidades
- 3 Heurística Lagrangiana para DC-MST
- 4 Implementación y Análisis Experimental

Algoritmo Fuerza Bruta

Idea principal: Explorar todas las combinaciones posibles de aristas que puedan formar un árbol generador válido respetando los límites de grado.

- Generar todos los subconjuntos de $|V| - 1$ aristas.
- Para cada subconjunto:
 - Verificar si forma un árbol (conexo y acíclico)
 - Comprobar que ningún vértice excede su límite de grado
 - Calcular el costo total
- Seleccionar la solución de menor costo

Por tanto, la complejidad temporal total del algoritmo es:

$$\mathcal{O}\left(\binom{m}{n-1} \cdot n\right)$$

Reducción del Árbol de Oportunidades

Idea principal: Aprovechar propiedades estructurales del árbol generador óptimo para reducir el grafo antes de construir la solución.

- Se identifican aristas y vértices que necesariamente deben o no pueden estar en la solución óptima (lemas estructurales).
- Se eliminan aristas innecesarias y se fijan otras en la solución parcial.
- Esto reduce el tamaño del problema y acelera la construcción del árbol final.

Ejemplo de reducción:

- Toda arista incidente a un vértice hoja debe estar en la solución.
- Ninguna arista entre dos vértices de grado máximo 1 puede estar en la solución si $|V| > 2$.
- Si un vértice v de grado 2 conecta dos vértices u y w , si no existe un camino entre u y w que no pase por v entonces tanto $\{u, v\}$ como $\{u, w\}$ deben pertenecer a la solución.

Esquema del algoritmo:

- 1 Aplicar la reducción estructural al grafo ($O(n^3)$).
- 2 Construir un árbol generador usando un algoritmo tipo Kruskal modificado para respetar los límites de grado.
- 3 Mejorar la solución con técnicas de intercambio de aristas (1-opt y 2-opt).

Complejidad:

- Reducción: $O(n^3)$
- Construcción inicial: $O(n)$
- 1-opt: $O(n)$, 2-opt: $O(n^2)$
- **Total:** $O(n^3)$

Ventaja: Permite obtener soluciones de alta calidad en tiempo polinomial, viable para instancias grandes.

Ejemplo

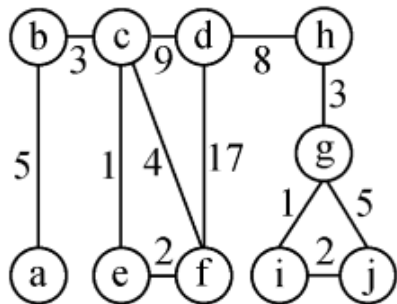


Figura 1: Grafo Original

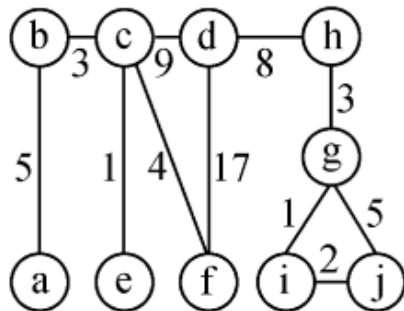


Figura 2: Grafo al quitarle la arista $\{e,f\}$

Ejemplo

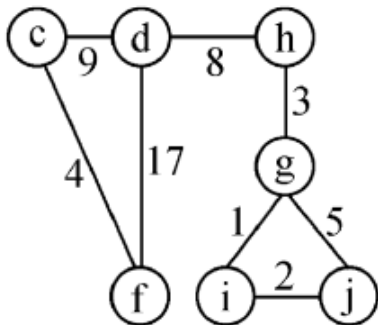


Figura 1: Grafo G

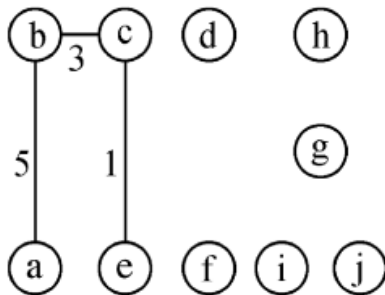


Figura 2: Grafo T^*

Ejemplo

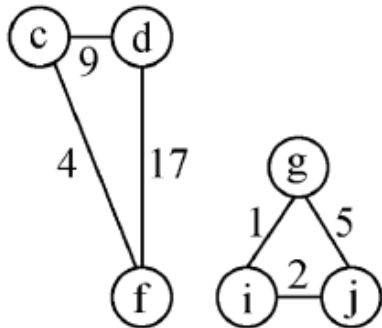


Figura 1: Grafo G

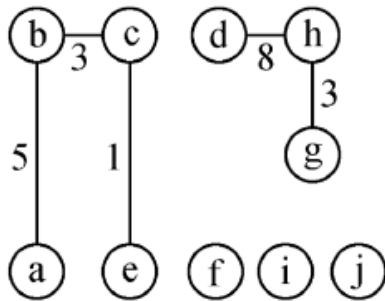


Figura 2: Grafo T*

Ejemplo

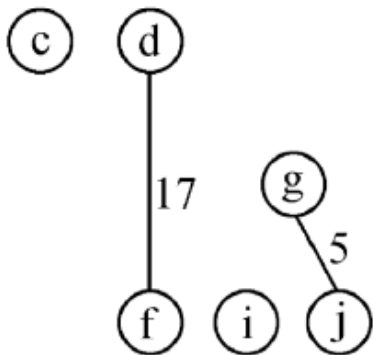


Figura 1: Grafo G

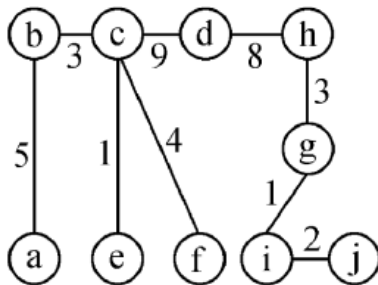


Figura 2: Grafo T*

Heurística Lagrangiana: Idea Principal

Idea principal: Relajar las restricciones de grado usando multiplicadores Lagrangianos, transformando el problema en uno de árbol generador clásico con costos modificados.

- Se obtiene una cota inferior resolviendo el MST con costos ajustados.
- Se usan multiplicadores para penalizar violaciones de grado y se actualizan con el método del subgradiente.
- Se construye una solución factible con un algoritmo tipo Kruskal modificado (KRUSKALX) y técnicas de mejora local.

Componentes principales:

- 1 Relajación Lagrangiana y método del subgradiente
- 2 Construcción heurística con prevención de infactibilidad
- 3 Definición de problema restringido para acelerar el proceso
- 4 Mejora local por intercambio de aristas

Heurística Lagrangiana: Esquema y Complejidad

Esquema del algoritmo:

- 1 Relajar restricciones de grado y resolver MST con costos modificados
- 2 Actualizar multiplicadores Lagrangianos por subgradiente
- 3 Construir solución factible con KRUSKALX y mejorarla localmente
- 4 Repetir hasta convergencia o máximo de iteraciones

Complejidad:

- Por iteración: $O(n^2 m' + m' \log m')$, donde m' es el número de aristas en el problema restringido
- Total: $O(K \cdot (n^2 m' + m' \log m'))$, con K iteraciones
- Para grafos completos: $O(Kn^3 \log n)$

Ventajas:

- Proporciona cotas de calidad y soluciones factibles eficientes
- Escalable a instancias grandes (miles de vértices)

Resultados de Escalabilidad

Límite de Fuerza Bruta: El algoritmo solo es viable hasta $n \approx 15$ vértices. Para $n = 20$, el tiempo de ejecución excede los 61 segundos (timeout).

Escalabilidad de las Heurísticas:

- **DCMST:** Escalable hasta $n \approx 960$ vértices (3.76 segundos). Falla en timeout para $n = 1920$.
- **Lagrange:** Escalable hasta $n \approx 480$ vértices (0.86 segundos). Falla en timeout para $n = 960$.

Observaciones:

- DCMST muestra mejor escalabilidad práctica que Lagrange
- El comportamiento de DCMST se ajusta razonablemente a su complejidad teórica $O(n^3)$
- Lagrange presenta comportamiento más errático debido a la naturaleza iterativa del subgradiente

Evaluación sobre 100 instancias de prueba:

Algoritmo	OK	FAIL	Resultado
Brute Force	100	0	[OK]
DCMST	95	5	[FAIL]
Lagrange	100	0	[OK]

Conclusiones:

- **Brute Force:** Garantiza optimalidad, pero solo viable para instancias pequeñas ($n \leq 15$)
- **DCMST:** Excelente escalabilidad y 95% de exactitud, ideal para instancias grandes
- **Lagrange:** 100% de exactitud con buena escalabilidad, mejor balance calidad-tiempo